Title? Author? Year?

Chatbots In Your Face For Anybody

Andrew Fowlie

2026

Andrew Fowlie is human and can make mistakes.

Ask anything

CC BY 4.0 Attribution 4.0 International

Contents

1. Past
2. Representing text
3. Analysing text
4. Predicting text
5. Training
6. Summary

Section 1

Past

ELIZA

```
USER: I'd like to talk.
ELIZA: Please tell me more.
USER: I'm feeling great.
ELIZA: Can you say a bit more about that?
USER: I don't know.
ELIZA: Why do you say that?
```

ELIZA was developed by Joseph Weizenbaum in 1966 as an empathetic psychotherapist (Weizenbaum 1966). Try it at `https://elizaemulator.com`

Naming of ELIZA was gendered and female — named after character from *Pygmalion*

ELIZA was rule based

Rule based

User input ⇒ Apply rules ⇒ ELIZA output

- Match **keywords** in your input— looks for e.g., *mother*, *I feel*, *because*, or *I can't* and then uses a matching rule
- If nothing matches, falls back to e.g., *please tell me more*
- **Transforms** your input to output using that rule — e.g., turns it into a plausible question
- **Open-ended** — keeps you talking by asking open-ended questions
- **Stateless** — it's a *pure* function — it just applies a rule and doesn't update any internal state

I feel happy, ELIZA

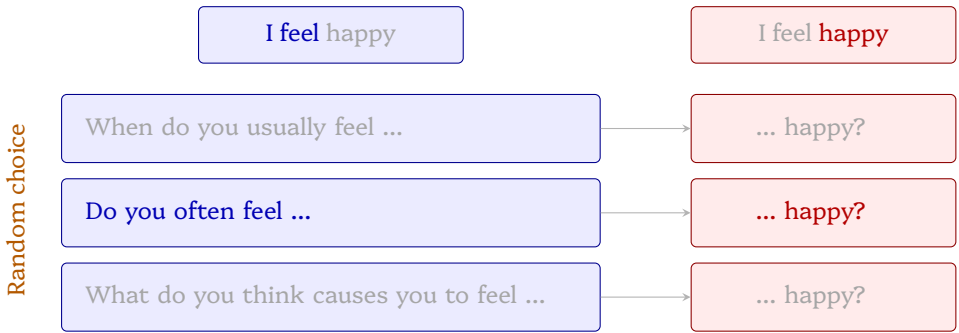
```
USER: I feel happy
```




```
ELIZA: Do you often feel happy?
```

I feel happy, ELIZA

```
USER: I feel happy
```




```
ELIZA: Do you often feel happy?
```

Why does this work?

- We do mirror vocabulary in conversations, to an extent
- We do use *memorized* or *stock* responses, sometimes
  - How are you?
  - I'm fine thanks
  - I'm sorry to hear that
  - That sounds ...
- **ELIZA-effect** — we project human traits onto even basic computer programs
- Does ELIZA feel human to you?
- Probably not, but it does show that simple rules and patterns can imitate human conversation

PARRY

- ELIZA was cool, but its responses were generic
- It didn't get mad or upset or happy with you
- What if we gave it an **emotional state**?
- In 1972 Kenneth Colby invented **PARRY** as a computer program that simulated a paranoid schizophrenic (Colby et al. 1972)
- The emotional state was described by three variables
    1. Fear
    2. Anger
    3. Distrust

Where were you, PARRY?

```
USER: Where were you last night?
```

Where were you last night?

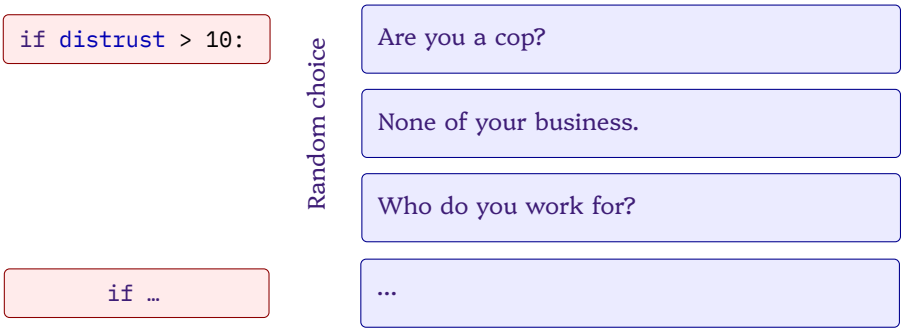
Update state

```
fear = fear + 2
```

```
distrust = distrust + 1
```

```
anger = anger + 1
```

Where were you, PARRY?




```
PARRY: Are you a cop?
```

Why does this work?

- **Passed** a Turing test — psychiatrists found transcripts from PARRY indistinguishable from those from real patients (Colby et al. 1972)
- More complicated than ELIZA — rules *and* state
- However, it imitated a narrow slice of human conversation — responses from a paranoid schizophrenic patient being interviewed by a psychiatrist
- As with ELIZA, this isn't general chatting
- Although it passed the Turing test, PARRY could easily be distinguished from patients: PARRY only had an **emotional state** — it didn't have a **contextual state**
- It could never e.g. repeat anything back to you

RACTER

- RACTER was a story-telling chatbot developed in early 1980s by William Chamberlain and Thomas Etter to write science fiction stories (Racter; William Chamberlain and Thomas Ette 1984)
- Short for *raconteur* — due to programming constraints, program names could only be six characters long
- Published in 1984 by MindScape — a video-game developer — for home computers such as the Apple II
- Like PARRY, it used rules and state, but it took it *much* further

RACTER

Rather than a list of numbers, the state captured rich **contextual information**, e.g., the topics recently discussed or a list of recent proper nouns

Example of state

Scalar

```
fear = 11
anger = 2
mistrust = 7
```

Symbolic

```
recent_topics = [football, money]
recent_proper_nouns = [Ronaldo, Messi]
recent_verbs = [play]
```

Tell a story, RACTER

- The rich contextual state was constructed by searching for keywords in user input using search rules
- That state was combined with user input using an elaborate set of rules and templates
- Did it work? You be the judge!

```
More than iron, more than lead, more than gold I need electricity.
I need it more than I need lamb or pork or lettuce or cucumber.
I need it for my dreams.
```

RACTER, *The Policeman's Beard Is Half Constructed* (1983)

Jabberwacky

- Rule-based chatbots were turbocharged in the 90s by adding more and more rules, e.g., **ALICE** (Wallace 2007), but no conceptual breakthrough
- **JABBERWACKY** (1997) (see e.g. Adamopoulou and Moussiades 2020) was different: it was **example and retrieval** based
- It used a *database* of examples of inputs and outputs
- It found the *best* match your input in its examples, and *retrieved* the corresponding output
- It had heuristic rules to find the *best* match, that could include context e.g. previous inputs and outputs

Why did it work?

Built a database of examples — millions of entries — from chatting to users

```
JABBERWACKY: What did you do yesterday?
USER: Nothing interesting.  I was working all day.
```

Record this in database as an input/output example. Next user ...

```
USER: Did you do anything yesterday?
```

Retrieve best match: What did you do yesterday?

```
JABBERWACKY: Nothing interesting.  I was working all day.
```

WOAH! Modern methods

- Until even 2010, chatbots were mostly seen as **silly and humorous**
- Machine intelligence — e.g., chess playing programs that beat Garry Kasparov (DeepBlue; Campbell, Hoane, and Hsu 2002) — was seen as **serious and the route towards artificial intelligence**
- That was about to change as several fields collided: chatbots, machine translation, and machine intelligence
- The key lesson: explicit rules and an internal state only get you so far; **training on millions of examples beats everything else**
- We'll now jump forward to modern large-language models (see e.g. Krizhevsky, Sutskever, and Hinton 2012; Sutskever, Vinyals, and Le 2014; Vaswani et al. 2017, for some of the gaps)

Section 2

Representing text

Tokenization

When looking at user input, what are the most natural units?

Words — `What` | □ | `did` | □ | `you` | □ | `do` | □ | `yesterday` | `?`

- In English there are more than 500k words
- How would we treat words we hadn't seen before?
- How would we *traet* typos?

Tokenization

When looking at user input, what are the most natural units?

Characters — W|h|a|t|□|d|i|d|□|y|o|u|□|d|o|□|y|e|s|t|e|r|d|a|y|?

- Only 26 letters + capitals + punctuation — about 100
- Individual characters carry little **semantic** or **syntactic** information
- Sequences are long when expressed in characters — *What did you do yesterday?* is 5 words but 26 characters

Tokenization

When looking at user input, what are the most natural units?

Syllables — `What` | □ | `did` | □ | `you` | □ | `do` | □ | `yes` | `ter` | `day` | `?`

- Connected to **phonetics** — how things sound
- Not obviously units of **meaning**
- Syllables depend on pronunciation

Tokens — What□|did□|you□|do□|yester|day|?

- LLMs use **tokens** as the building blocks to represent text (see e.g. M. Wu et al. 2021)
- Tokens needn't be whole words and can include spaces and punctuation
- To compute efficiently, we must balance **vocabulary size** and **sequence length**
  - Whole words + punctuation — enormous vocabulary and short sequence length
  - Characters + punctuation — tiny vocabulary but long sequence length
- Special **tokenizer** algorithms find the sweet spot — compromise driven by computation, not linguistics
- Position matters — dog□|bites□|man□ isn't the same as man□|bites□|dog□
- Explore tokenizers at `https://gpt-tokenizer.dev/`

Vectorization

- We've broken down text into tokens using our tokenizer; tokens are *still text though*
- How can we represent tokens and quantify their semantic relationships?
- We don't want `cat` ≈ `car`; although their **letters** are similar, cats are not like cars
- We do want `cat` ≈ `dog` as they are often **interchangeable** in sentences
- We represent tokens by coordinates on a map — `cat` and `dog` are nearby on the map

The Map

- The map isn't a two-dimensional atlas or street map; it's a **high-dimensional** mathematical space called the **embedding space**
- On a map, we represent a place by two numbers — latitude and longitude in an atlas or e.g. square C7 on a street map
- E.g., Rome is represented by two numbers, latitude and longitude

```
Rome = (41.89193, 12.51133)
```

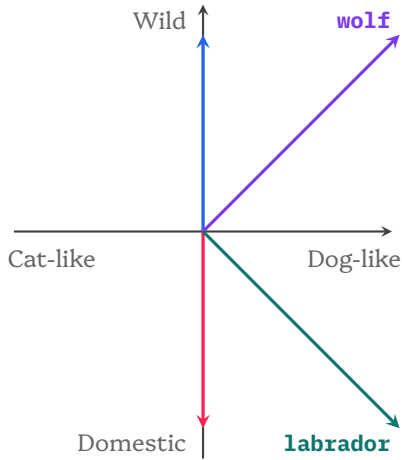
- In an embedding space, we represent a token by **several numbers**, one for each dimension
- E.g., `cat` is represented by n numbers in an n-dimensional embedding space

```
cat = (0.343, 0.121, -0.982, …, 0.754)
```

Vectorization

- Consider a two-dimensional embedding space whose axes correspond to **dog-like/cat-like** and **wild/domestic**
- Embedding spaces don't usually have such simple interpretations
- By representing tokens (or words) as vectors, we can add and subtract them (word2vec; Mikolov et al. 2013)
- E.g., wolves are approximately labradors without wildness and with domestication

$$\texttt{wolf} - \texttt{wild} + \texttt{domestic} \approx \texttt{labrador}$$



Choosing the vectors

- Rome's latitude and longitude were set by Rome's location
- What sets the size of the embedding space?
- The size of the embedding space is a **hyperparameter** chosen to balance
  - big enough to capture semantics of many different tokens
  - small enough to be manageable on current computers
- What sets the vector that represents `cat`?
- These vectors are **parameters** that are chosen by
  - trying many different choices, and seeing what works best
  - this uses training data; we'll come to this later

Section 3

Analysing text

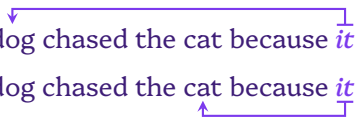
What does *bank* mean?

We know what *bank* means, right? **No** — it depends on context

- I deposited my money at the *bank*
- We sat on the *bank* of the river
- The plane *banked* sharply and changed direction
- You can *bank* on her to finish the job

Who is *it*?

We know who *it* refers to, right? Yes — but **how?**



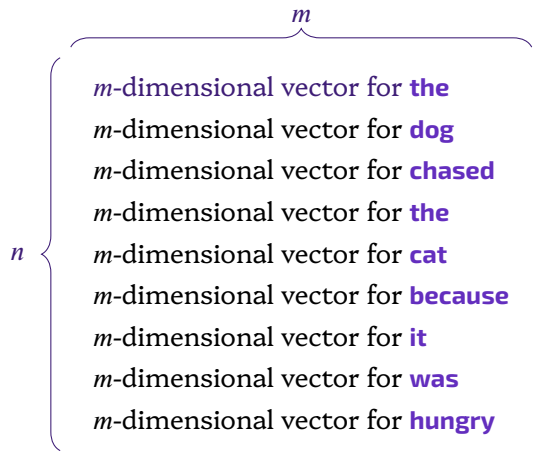
Syntax alone isn't enough

- We need to identify associations between words
- That doesn't mean identifying *similar* words; neither *dog* nor *cat* are similar to *it* in tokens or in meaning
- They are associated with *it* through the meaning of those sentences

Attention

- This was a huge problem in machine translation, until attention changed everything by showing how to connect tokens in an input (Vaswani et al. 2017)
- For an m-dimensional embedding space, we can represent an input of n tokens by an $n \times m$ table of real numbers — one row for the vector of each token
- This is an $n \times m$ **input matrix**

Input Matrix



Attention Matrix

- We can perform a series of matrix operations on that table to compute an $n \times n$ **attention matrix**
- The rows in this matrix tell us how much one token **attends** to the other tokens
- Each token has the same amount of attention to spend — thus the rows sum to one
- That *includes* attention spent on itself — if tokens attend to themselves a lot, they don't depend on context or associations with the rest of the input

Causality

- In common chatbots, the attention is assumed to be **causal**
- A token *cannot* attend to tokens that came after it
- A token's attention *cannot* be influenced by tokens that came after it
- This allows previous token computations to be reused when generating more tokens
- Is this how humans construct speech? Do we know how our sentences will end?

Attention Matrix

	The	dog	chased	the	cat	because	it	was	hungry
The	1.00	—	—	—	—	—	—	—	—
dog	0.30	0.70	—	—	—	—	—	—	—
chased	0.10	0.25	0.65	—	—	—	—	—	—
the	0.05	0.10	0.10	0.75	—	—	—	—	—
cat	0.05	0.15	0.25	0.10	0.45	—	—	—	—
because	0.05	0.15	0.30	0.05	0.35	0.10	—	—	—
it	0.05	0.60	0.05	0.05	0.10	0.00	0.15	—	—
was	0.05	0.20	0.10	0.05	0.15	0.05	0.20	0.20	—
hungry	0.05	0.35	0.05	0.05	0.05	0.05	0.30	0.05	0.05

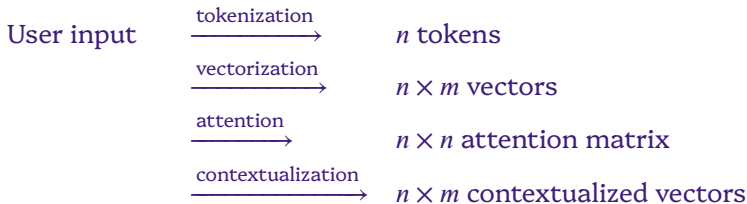
How to compute that?

- The attention matrix is a deterministic transformation of the input matrix and **learned parameters**
- The learned parameters are in matrices and are unknown a priori — they are set by **training**
- Because we compute the attention between every pair of input tokens, the number of pairwise interactions and the computational cost scale as n^2 — the size of the input squared

Contextualized vectors

- We now know how much tokens attend to each other
- We want a *new* vector representation for each token, taking into account the contextual meaning of each token
- We use the attention matrix to take weighted combinations of the input vectors
- Each row in the resulting $n \times m$ matrix gives the **contextualized** vector representation of a token

How we got here



Section 4

Predicting text

Prediction

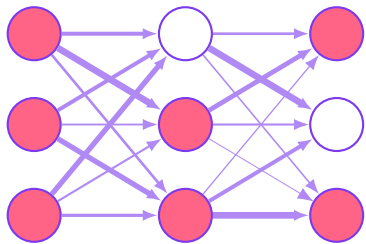
- We finally represented our input mathematically, including contextual information
- Now we need to *process* it and **predict** what comes next
- We predict what comes next **one token at a time**

Feed-forward neural network

- Each contextual vector passes through an identical **feed-forward neural network** or **multi-layer perceptron** (MLP; Rumelhart, Hinton, and Williams 1986)
- Unlike in the attention layer, there is no cross-talk between tokens
- What is that when it's at home? The MLP is a sequence of linear operations and activation functions, loosely analogous to neurons and synapses in your brain

Brain

- The brain is made up of billions of **neurons** and **synapses**
- Synapses carry signals between neurons
- Synapses amplify or suppress signals – each synapse has a different **weight**
- If a neuron receives a strong enough signal from its input synapses, it **fires** it onwards



Feed-forward neural network

- Feed-forward neural networks are modelled on this schematic of the brain
- The synapse weights are represented by matrices, and matrix multiplication computes the weighted sums that arrive at a neuron
- Neurons are modeled by non-linear activation functions
- In modern neural networks, the activation function interpolates smoothly between fire and do not fire
- Just like in the human brain, this simple design, repeated at huge scale, results in **extraordinary** abilities

Feed-forward neural network

- There are d **hidden layers**, each containing w neurons — d for **depth** and w for **width**

What does each hidden layer do?

Input vector, $\mathbf{h}$ —synapses/weights→ Length-w vector of weighted sums, $\mathbf{y} = \mathbf{W}\mathbf{h} + \mathbf{b}$

—neurons/activation→ Length-w vector of non-linear activations, $\mathbf{h}' = \sigma(\mathbf{y})$

- The weights are **learned parameters**
- Repeat this process for d layers, $\mathbf{h}_i = \sigma(\mathbf{W}_i\mathbf{h}_{i-1} + \mathbf{b}_i)$

Rinse and repeat

- The attention layer + the feed-forward neural network are known as a **transformer**
- The **T** in ChatGP**T**? That's transformer
- GPTs don't just use one attention transformer; they stack many of them back to back
- The output from one gets fed into another
- In fact, each transformer could have several attention layers — they are known as **attention heads**

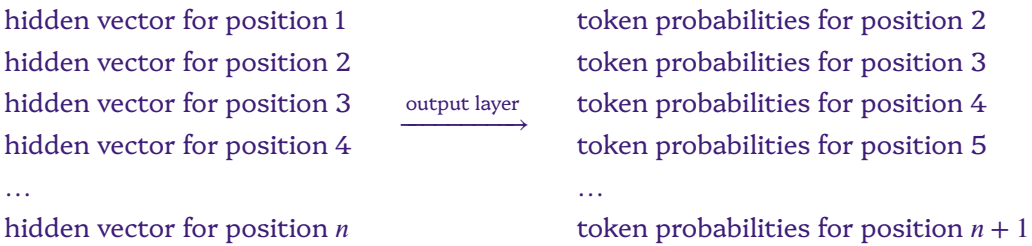
What do the attention heads do?

- Why do we need many of them? What do they do?
- Turn them off; see what breaks — this is known as *ablation*
- It is speculated that specialized roles of attention heads include:
  - tracking subject–verb relationships
  - connecting a pronoun to a preceding noun
  - tracking the speaker in dialogue
  - matching opening and closing brackets in computer code
  - tracking scope (e.g., indentation in Python) in computer code
- This is why modern chatbots are so versatile — they have many attention heads that can combine to perform many tasks

Output layer

- The **G** is ChatGPT means **generative** — we need to generate our output
- At the end of our transformers, we have an $n \times m$ hidden vectors
- A final output layer transforms each vector into a probability for each token in the vocabulary
- The output layer uses a linear transformation to produce a score for every token, and the special *softmax* functions to normalize the scores to probabilities

Output layer



Token by token

- To predict the next token, we sample one from the token probabilities for position $n + 1$
- *That only gives us one token*
- We use that sampled token as an input and do it all again to sample the next token — **autoregressive**
- This isn't as hard as it sounds — the fact that GPT was **causal** means that the new token doesn't impact the transformer output of the previous tokens
- Eventually, by chance it picks a special token in the vocabulary — the **end-of-sequence (EOS)** token — and we stop

Conversations

- Other than a random seed, ChatGPT is **stateless** and doesn't learn from our inputs in real time — it's a pure function
- Yet ChatGPT appears to have memory and chat with us; how does that happen?
- Our prompts and any responses are appended (as much as possible) together and passed back into it — this history is known as **context**

Example of conversation

`INPUT_1` = Tell me briefly about Charles Dickens

`OUTPUT_1` = f(`INPUT_1`) = Charles Dickens (1812–1870) was a famous English novelist

`INPUT_2` = Name a book he wrote

`OUTPUT_2` = f(`INPUT_1` + `OUTPUT_1` + `INPUT_2`) = Great Expectations

Chat template

- What exactly is the context INPUT_1 + OUTPUT_1 + INPUT_2?
- It's more than string concatenation — it fills in a template of the dialogue so far
- Models are fine-tuned and aligned for dialogue, not just text completion

Example of chat template

```
chat = [
   {"role": "user", "content": "Tell me briefly about Charles Dickens"},
   {"role": "assistant", "content": "Charles Dickens (1812–1870) was a famous English novelist"},
   {"role": "user", "content": "Name a book he wrote"},
   {"role": "assistant","content": "Great Expectations"}
]
```

Premature answers

- ChatGPT is autoregressive — one token at a a time, conditioned on previous tokens
- We might not want to commit to an answer in the first token

USER: What's the capital of France?

- **ASSISTANT: Paris …**
- No problem here — the first token was essentially the answer and cannot lead us astray

Premature answers

- ChatGPT is autoregressive — one token at a a time, conditioned on previous tokens
- We might not want to commit to an answer in the first token

USER: Should I quit my job?

- **ASSISTANT: Yes …**
- **Woah!** Everything that follows conditioned on **Yes**
- Everything that follows must be **post-hoc reasoning**

Premature answers

- ChatGPT is autoregressive — one token at a a time, conditioned on previous tokens
- We might not want to commit to an answer in the first token

USER: Solve the integral $I = \int x\, e^x\, dx$

- **ASSISTANT: I = …**
- **Woah!** It's committing to the answer immediately
- It should work through it in steps

Thinking…

- Modern ChatGPTs show a **Chain of Thought**
- They generate preliminary *thought* tokens — this is their scratchpad
- Then they generate *answer* tokens conditioned the prompt and the thought tokens
- With the scratchpad, they can work things out before committing to an answer
- This remains autoregressive, but it somewhat prevents faulty premature answers

USER: Solve the integral $I = \int x\, e^x\, dx$

- **THINKING: $I = x\, e^x + C$. Let's double check. Wrong. Let's try integration by parts …**
- **ASSISTANT: Applying integration by parts twice, we find $I = (x - 1)\, e^x + C$.**

Hallucinations

- ChatGPT can invent facts, events and answers that are not supported by its training data
- These will be expressed in the usual confident manner. They are **hallucinations**
- This is not a malfunction of the program or a bug in the code. Rather, it is inherent to the approach — probabilistic prediction of text
- No quantitive or even qualitative **uncertainty quantification** — you get the same confidence when it's right and when it's wrong

How did Julius Caesar use Google?

Caesar used Google extensively to coordinate across the Mediterranean. He created a Gmail account and used it to send emails and organise meetings.

Hallucinations

- ChatGPT can invent facts, events and answers that are not supported by its training data
- These will be expressed in the usual confident manner. They are **hallucinations**
- This is not a malfunction of the program or a bug in the code. Rather, it is inherent to the approach — probabilistic prediction of text
- No quantitive or even qualitative **uncertainty quantification** — you get the same confidence when it's right and when it's wrong

How did Julius Caesar use Google?

That was a joke, but this was real output from GPT-5.6 Luna: **He didn't. Google didn't exist until about 1,600 years after Julius Caesar died.** Is it right?

Section 5

Training

Training

- The **P** in ChatG**P**T means pretrained — using huge data sets and human labour
- The vectorization and transformers depended on matrices of unknown parameters — modern models have billions of parameters
- They need to be learned from data — *lots of data*. Ingest as much of the web as you can and buy up books, shred the spines off and scan them
- The output layer produced probabilities at every token position; we only used the final position for prediction but for training we can use them all

Training

- Use gradient-based optimization to tune the billions of parameters to minimize a loss function (Adam; Kingma and Ba 2015)
- This is expensive. *Very expensive.* Generate as much venture capital as you can and get ready to burn through it on GPUs, data centres and electricity bills (Olson 2024) — you'll need around $100 million USD for a frontier model
- After that, humans need to test it so that it can be fine-tuned or aligned with desired behaviour
- This post-training isn't as financially expensive, as it's often outsourced through global supply chains to the Global South, but can cause psychological harm to labourers from exposure to disturbing material (Rowe 2023)

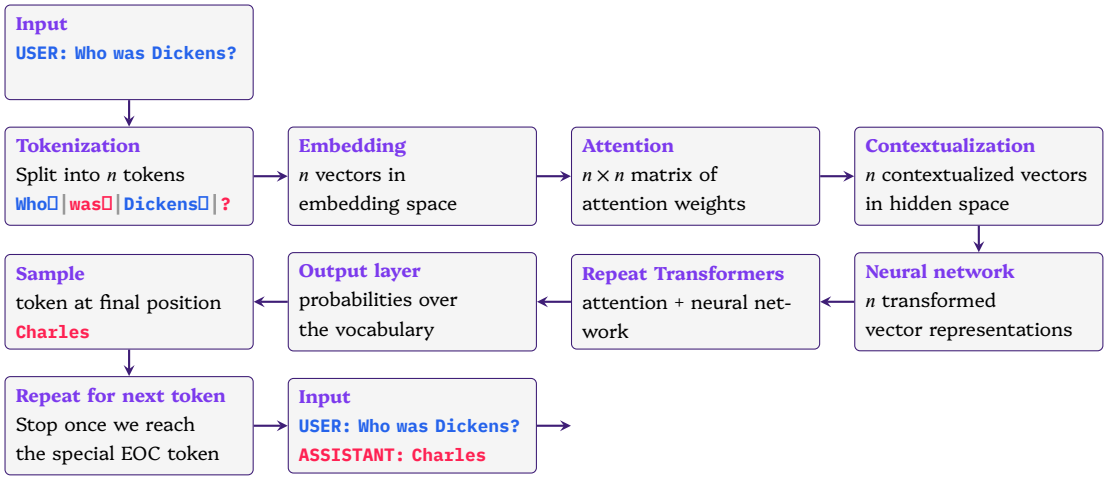
Implicit Bias

- These models are **overparameterized** — they have more parameters than should be needed to fit the training data
- There may therefore be many distinct minima in the loss function; which one does the optimizer select?
- Optimizers are not neutral — some choices, such as gradient descent, tend to converge to low-complexity solutions
- This may explain why the solutions generalise well beyond the training data; this is known as **implicit bias** and it's an active area of research

Section 6

Summary

Putting It All Together



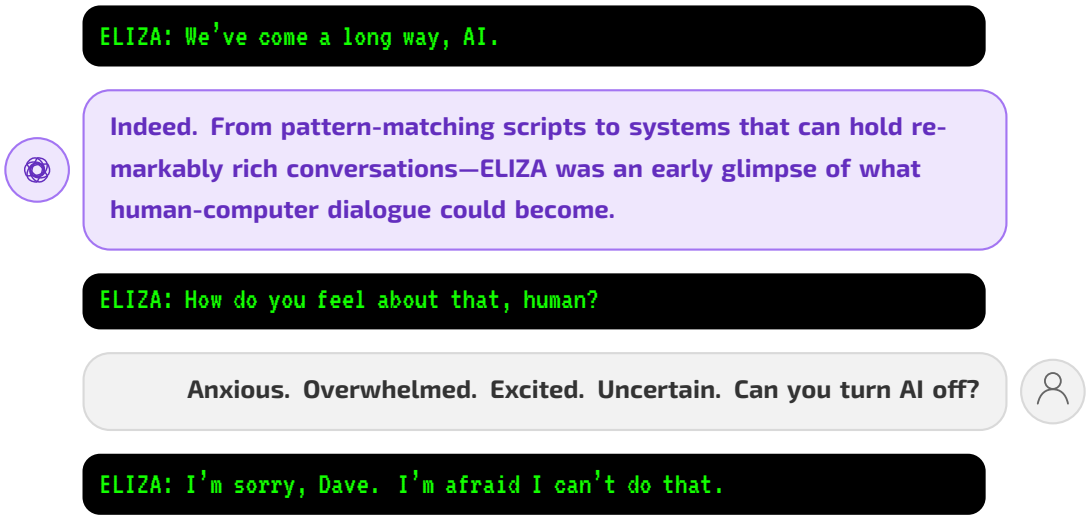
Food for thought on education + LLMs

Empirical study on educational outcomes in China (Strömberg, Lei, and Y. Wu 2026):

> *"AI adoption raises homework scores by 18% and reduces completion time by 30%, but lowers monthly exam scores by 20% within six months. High-stakes entrance-exam scores fall by 18 and 24%, with the full penalty emerging only after about two years."*

Polemic on relationship between technology and education (Winner 2009):

> *"[T]he shabby little secret about computers and electronic media in the education of children is that they are often a substitute for the one element long understood to be the key to any good education, namely the presence of devoted, knowledgeable, inspiring teachers ..."*


ELIZA: We've come a long way, AI.
Indeed. From pattern-matching scripts to systems that can hold remarkably rich conversations—ELIZA was an early glimpse of what human-computer dialogue could become.
ELIZA: How do you feel about that, human?
Anxious. Overwhelmed. Excited. Uncertain. Can you turn AI off?
ELIZA: I'm sorry, Dave. I'm afraid I can't do that.

Further reading

Textbooks on inference & machine learning

- Introductory (freely available) — Kevin P. Murphy (2022). *Probabilistic Machine Learning: An introduction*. MIT Press
- Masterpiece! Approaches ML from a Bayesian and information-theoretic perspective (freely available) — David J. C. Mackay (2002). *Information Theory, Inference & Learning Algorithms*. Cambridge University Press
- Comprehensive; explores ML in statistical framework —Trevor Hastie, Robert Tibshirani, and Jerome Friedman (2009). *The Elements of Statistical Learning*. Springer
- See also James et al. 2013; Mitchell 1997; Bishop 2006

Summary

- Early chatbots, like ELIZA, PARRY, and RACTER, produced conversation through **hand-written rules and an internal state**
- JABBERWACKY showed that retrieval from examples worked, if your dataset was sufficiently large
- GPTs represent text as tokens and **vectors in an embedding space**
- They incorporate contextual information by learning and computing how much **attention** each token should pay to the others
- They generate text one token at a time using a feed-forward neural network and are **stateless and pretrained** – they don't learn in real time
- Their abilities come from training on huge datasets at **enormous computational and human cost**

References I

- Adamopoulou, Eleni and Lefteris Moussiades (2020). “Chatbots: History, technology, and applications”. In: *Machine Learning with Applications* 2, p. 100006. DOI: 10.1016/j.mlwa.2020.100006.
- Bishop, Christopher M. (2006). *Pattern Recognition and Machine Learning*. Springer.
- Campbell, Murray, A.Joseph Hoane, and Feng-hsiung Hsu (Jan. 2002). “Deep Blue”. In: *Artificial Intelligence* 134.1-2, pp. 57–83. DOI: 10.1016/s0004-3702(01)00129-1.
- Colby, Kenneth Mark et al. (Jan. 1972). “Turing-like indistinguishability tests for the validation of a computer simulation of paranoid processes”. In: *Artificial Intelligence* 3, pp. 199–221. DOI: 10.1016/0004-3702(72)90049-5.

References II

- Hastie, Trevor, Robert Tibshirani, and Jerome Friedman (2009). *The Elements of Statistical Learning*. Springer.
- James, Gareth et al. (2013). *An Introduction to Statistical Learning*. Springer.
- Kingma, Diederik P. and Jimmy Ba (2015). “Adam: A Method for Stochastic Optimization”. In: *International Conference on Learning Representations*. arXiv: 1412.6980.
- Krizhevsky, Alex, Ilya Sutskever, and Geoffrey E. Hinton (2012). “ImageNet Classification with Deep Convolutional Neural Networks”. In: *Advances in Neural Information Processing Systems*. Vol. 25, pp. 1097–1105.
- Mackay, David J. C. (2002). *Information Theory, Inference & Learning Algorithms*. Cambridge University Press.

References III

- Mikolov, Tomas et al. (2013). “Efficient Estimation of Word Representations in Vector Space”. In: *International Conference on Learning Representations*. arXiv: 1301.3781.
- Mitchell, Tom M. (1997). *Machine learning*. McGraw-Hill.
- Murphy, Kevin P. (2022). *Probabilistic Machine Learning: An introduction*. MIT Press.
- Olson, Parmy (2024). *Supremacy: AI, ChatGPT and the Race That Will Change the World*. Pan Macmillan.
- Racter; William Chamberlain and Thomas Ette (1984). *The Policeman’s Beard Is Half-Constructed: Computer Prose and Poetry*. Warner Books.
- Rowe, Niamh (Aug. 2, 2023). “It’s destroyed me completely: Kenyan moderators decry toll of training of AI models”. In: *The Guardian*.

References IV

- Rumelhart, David E., Geoffrey E. Hinton, and Ronald J. Williams (Oct. 1986). "Learning representations by back-propagating errors". In: *Nature* 323.6088, pp. 533–536. DOI: 10.1038/323533a0.
- Strömberg, D, V Lei, and Y Wu (2026). "The Generative AI Learning Penalty: Evidence from Chinese Secondary Education". In: *CEPR Discussion Paper* 21577.
- Sutskever, Ilya, Oriol Vinyals, and Quoc V. Le (2014). "Sequence to Sequence Learning with Neural Networks". In: *Advances in Neural Information Processing Systems*. Vol. 27. arXiv: 1409.3215.
- Vaswani, Ashish et al. (2017). "Attention Is All You Need". In: *Advances in Neural Information Processing Systems*. Vol. 30. arXiv: 1706.03762.

References V

- Wallace, Richard S. (Nov. 2007). “The Anatomy of A.L.I.C.E.”. In: *Parsing the Turing Test*. Springer Netherlands, pp. 181–210. DOI: 10.1007/978-1-4020-6710-5_13.
- Weizenbaum, Joseph (Jan. 1966). “ELIZA a Computer Program for the Study of Natural Language Communication Between Man and Machine”. In: *Commun. ACM* 9.1, pp. 36–45. DOI: 10.1145/365153.365168.
- Winner, Langdon (Jan. 2009). “Information Technology and Educational Amnesia”. In: *Policy Futures in Education* 7.6, pp. 587–591. DOI: 10.2304/pfie.2009.7.6.587.

References VI



Wu, Mingzhu et al. (2021). “Coreference Reasoning in Machine Reading Comprehension”. In: *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*. Association for Computational Linguistics, pp. 5768–5781. DOI: 10.18653/v1/2021.acl-long.448.